# Laboratório 3: Rodando a API de Intent Classifier na sua máquina e em um container Docker

IMD3005 - MLOps · 2026.2

## Objetivo e entrega

Neste laboratório você vai rodar, na sua própria máquina, a API de classificação de intenções construída nas aulas, conectada ao seu modelo no Weights & Biases Forge e ao seu banco no MongoDB Atlas. Você termina quando a rota `POST /predictions` responder 200 no Swagger usando as suas credenciais.

O que você entrega (um único arquivo `.zip` ou `.pdf`):

1. Um print do Swagger mostrando a requisição `POST /predictions` com o resultado código 200, feita com o seu `.env`.
2. O conteúdo do arquivo `controller.py` que você utilizou.
3. Um print dos logs do container Docker (`docker logs -f mlops-container`) com a requisição `POST /predictions` registrada com `200 OK`.

O arquivo `.env` contém senhas e não faz parte da entrega, e não pode ser commitado nem aparecer em nenhum print.

## Antes de começar

Você precisa de três contas gratuitas e de algumas ferramentas instaladas. Crie as contas antes da aula de laboratório, para não perder tempo.

| Item | Para que serve | Onde |
|---|---|---|
| Git | baixar o código do repositório | git-scm.com |
| Conda (Miniconda) | criar o ambiente Python 3.11 isolado | docs.conda.io |
| Conta Weights & Biases | guardar o modelo treinado e a chave de API | wandb.ai |
| Conta MongoDB Atlas | banco onde as predições são gravadas | mongodb.com/cloud/atlas |
| Conta Google | treinar o modelo no Colab | colab.research.google.com |

Sistema operacional: Linux ou macOS. No Windows, use o WSL2 (o pacote `tensorflow-text` não costuma ter versão para Windows nativo).

## Parte 1: repositório e ambiente conda

Você precisa da versão mais recente do código (a que já inclui a integração com o MongoDB) e de um ambiente Python limpo.

1. Atualize o repositório. Se já clonou antes, entre na pasta e rode `git pull`. Se ainda não clonou, rode `git clone https://github.com/adaj/mlops-2026-2.git` e entre na pasta.
2. Crie o ambiente conda com Python 3.11: `conda create -n mlops-2026-2 python=3.11 -y`
3. Ative o ambiente: `conda activate mlops-2026-2`
4. Instale as dependências, na raiz do projeto: `pip install -r requirements.txt` (demora alguns minutos).
5. Confirme: `python -c "import tensorflow as tf, keras; print(tf.__version__, keras.__version__)"` deve mostrar `2.19.1` e `3.13.2`.

## Parte 2: seu modelo, seu banco e seu .env

A API precisa de quatro informações que são suas: a chave do W&B, o caminho do seu modelo, o nome do projeto e o endereço do seu banco. Siga a ordem abaixo.

### 2.1 Chave de API do W&B

1. Faça login no W&B e abra https://wandb.ai/authorize (ou: foto do perfil, User Settings, seção API Keys).
2. Copie a chave (começa com `wandb_v1_`). Guarde-a em local seguro: ela é uma senha.

### 2.2 Treinar o modelo no Colab

1. Abra o notebook IntentClassifier e use *Arquivo, Salvar uma cópia no Drive*.
2. No Colab, abra Secrets (ícone de chave na barra lateral) e crie o secret `WANDB_KEY` com a sua chave. Ative Acesso ao notebook.
3. Na aba Arquivos do Colab, envie o arquivo `confusion_examples.yml` (disponível na turma).
4. Execute as células em ordem: instalação, classe `IntentClassifier`, login no W&B e treinamento do modelo de confusão.
5. No W&B, abra o projeto `mlops-2026-2`, aba Artifacts, artefato `confusion-clf`. Na aba Files devem existir `model.keras` e `model_config.yml`.
6. Copie o nome completo do artefato, no formato `seu_usuario/mlops-2026-2/confusion-clf:v0`.

### 2.3 Banco no MongoDB Atlas

1. Crie um projeto e um cluster gratuito M0.
2. Em Database Access, crie um usuário e uma senha (evite caracteres especiais).
3. Em Network Access, adicione o seu IP. Em aula, `0.0.0.0/0` funciona, mas deixa o cluster aberto: use só com dados fictícios.
4. Em Connect, Drivers, Python, copie a *connection string* e troque `<db_password>` pela senha do passo 2.

### 2.4 Montar o .env

Na raiz do projeto, copie o modelo com `cp .env.example .env` e preencha:

```
WANDB_API_KEY=wandb_v1_...
WANDB_PROJECT=mlops-2026-2
WANDB_MODELS=seu_usuario/mlops-2026-2/confusion-clf:v0
MONGO_URL=mongodb+srv://usuario:senha@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

Sem espaços ao redor do `=`. Rode `git status`: o `.env` não pode aparecer na lista (ele está no `.gitignore`). Mais detalhes de cada variável estão no `README.md` do repositório.

## Parte 3: subir a API e testar no Swagger

1. Com o ambiente ativo e na raiz do projeto, rode: `uvicorn src.app:app --port 8000`
2. Aguarde a mensagem `Application startup complete.` Na primeira vez demora, porque a API baixa o seu modelo do W&B.
3. Abra http://localhost:8000/docs no navegador.
4. Clique em `POST /predictions`, depois em Try it out.
5. Preencha o corpo da requisição. Use o seu nome real no campo `nome`: ele identifica a autoria do seu print.

```
{
  "id": "lab-001",
  "nome": "Seu Nome Completo",
  "text": "eu não estou entendendo nada"
}
```

6. Clique em Execute. Role até *Server response*: o código deve ser 200, e o corpo deve trazer o resultado do modelo `confusion-clf` (a intenção e as probabilidades) e a sua mensagem com o `nome` que você digitou.
7. Confirme que o banco recebeu o registro: no Atlas, Browse Collections, banco `intent_classifier`, coleção `predictions`.

Se aparecer `422`, falta algum campo no corpo (envie `id`, `nome` e `text`). Para outros erros, consulte a tabela de solução de problemas do `README.md`.

## Parte 4: a mesma API dentro de um container Docker

Até aqui você montou o ambiente à mão: conda, Python 3.11 e dezenas de bibliotecas. O Docker empacota tudo isso, mais o seu código, em uma imagem: uma receita fechada que roda igual em qualquer máquina. Um container é uma imagem em execução. No fim desta parte, um único comando sobe a mesma API que você rodou com `uvicorn`.

### 4.1 Instalar o Docker

| Sistema | O que instalar | Como |
|---|---|---|
| Windows | Docker Desktop (com WSL2) | Baixe em docs.docker.com/get-started/get-docker, instale, mantenha marcada a opção de usar o WSL 2 e reinicie o computador. |
| macOS | Docker Desktop | Baixe a versão do seu chip (Apple Silicon ou Intel) e arraste para Aplicativos. |
| Linux | Docker Engine e plugin Compose | Siga o guia da sua distribuição no mesmo site. Depois rode `sudo usermod -aG docker $USER` e abra uma nova sessão, para usar `docker` sem `sudo`. |

No Windows e no macOS, abra o aplicativo Docker Desktop e espere o status indicar que o motor está rodando. Ele precisa ficar aberto enquanto você usa o Docker. Confira os termos de licença do Docker Desktop no site (uso pessoal e educacional costuma ser gratuito).

Verifique a instalação:

```
docker --version
docker compose version
docker run hello-world
```

O terceiro comando baixa uma imagem mínima e imprime `Hello from Docker!`. Se isso apareceu, o Docker está pronto.

Antes de seguir, pare a API local (Ctrl+C no terminal do `uvicorn`): o container vai usar a mesma porta 8000.

### 4.2 Escrever o Dockerfile

Na raiz do projeto, crie um arquivo chamado `Dockerfile` (exatamente assim, sem extensão) com este conteúdo:

```
FROM python:3.11-slim

WORKDIR /app

RUN useradd -ms /bin/bash appuser && chown -R appuser:appuser /app

RUN apt-get update && apt-get install -y build-essential libffi-dev && rm -rf
/var/lib/apt/lists/*

USER appuser
ENV PATH="/home/appuser/.local/bin:$PATH"

RUN pip install --no-cache-dir --upgrade pip

COPY --chown=appuser:appuser requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=appuser:appuser src ./src
ENV PYTHONPATH=/app

EXPOSE 8000

CMD ["uvicorn", "src.app:app", "--host", "0.0.0.0", "--port", "8000", "--log-level",
"info"]
```

O Dockerfile é lido de cima para baixo e cada instrução vira uma camada da imagem. O Docker guarda as camadas em cache e só refaz a partir da primeira que mudou. Por isso a ordem importa.

| Instrução | O que faz |
|---|---|
| `FROM python:3.11-slim` | Ponto de partida: um Linux enxuto já com Python 3.11, a mesma versão do seu ambiente conda. |
| `WORKDIR /app` | Define a pasta de trabalho dentro da imagem. Os comandos seguintes rodam nela. |
| `RUN useradd -ms /bin/bash appuser && chown -R appuser:appuser /app` | Cria um usuário comum (`appuser`) e o torna dono de `/app`. Rodar a API como administrador (root) é má prática de segurança. |
| `RUN apt-get update && apt-get install -y build-essential libffi-dev && rm -rf /var/lib/apt/lists/*` | Instala compiladores que alguns pacotes Python exigem. O `rm -rf` no final apaga a lista de pacotes do apt para a imagem ficar menor. |
| `USER appuser` | A partir daqui, tudo roda como `appuser`. |

| Instrução | O que faz |
|---|---|
| `ENV PATH="/home/appuser/.local/bin:$PATH"` | O pip instala programas como o `uvicorn` em `~/.local/bin`. Esta linha os torna encontráveis. |
| `RUN pip install --no-cache-dir --upgrade pip` | Atualiza o pip. O `--no-cache-dir` evita guardar os downloads e reduz o tamanho da imagem. |
| `COPY --chown=appuser:appuser requirements.txt .` | Copia só o `requirements.txt` primeiro, com `appuser` como dono. Assim o Docker reaproveita a instalação das dependências quando você muda apenas o código. |
| `RUN pip install --no-cache-dir -r requirements.txt` | Instala as dependências (TensorFlow, FastAPI, pymongo, wandb). É a etapa mais demorada. |
| `COPY --chown=appuser:appuser src ./src` | Copia o código da API para `/app/src`. Fica depois das dependências porque é o que mais muda. |
| `ENV PYTHONPATH=/app` | Faz o Python encontrar o pacote `src`, pois os imports do projeto começam com `from src.modules...`. |
| `EXPOSE 8000` | Documenta que a API escuta na porta 8000. Ela não abre a porta para fora; isso é feito no `docker-compose.yml`. |
| `CMD ["uvicorn", "src.app:app", ...]` | Comando que roda quando o container inicia, o mesmo `uvicorn src.app:app` de antes. `--host 0.0.0.0` é obrigatório: com o padrão (127.0.0.1) a API ficaria inacessível fora do container. |

### 4.3 Escrever o .dockerignore

Ao construir a imagem, o Docker envia a pasta do projeto para o motor do Docker. O arquivo `.dockerignore` lista o que não deve ser enviado. Crie-o na raiz, ao lado do `Dockerfile`:

```
.git
.env
__pycache__
*.pyc
wandb
**/models
.DS_Store
```

O ponto principal é o `.env`: ele guarda suas senhas e nunca deve entrar em uma imagem. As pastas `models` e `wandb` são downloads locais, grandes, que a API refaz sozinha dentro do container.

### 4.4 Escrever o docker-compose.yml

Subir um container com `docker build` e `docker run` exige muitas opções (porta, variáveis, nome). O Docker Compose guarda tudo isso em um arquivo. Na raiz, crie `docker-compose.yml`:

```
name: mlops-2026-2
services:
  mlops-container:
    container_name: mlops-container
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - WANDB_API_KEY=${WANDB_API_KEY}
      - WANDB_PROJECT=${WANDB_PROJECT}
      - WANDB_MODELS=${WANDB_MODELS}
      - MONGO_URL=${MONGO_URL}
    restart: unless-stopped
```

| Linha | O que faz |
|---|---|
| `name: mlops-2026-2` | Nome do projeto Compose. Aparece como prefixo nos nomes de imagem e rede criados. |
| `services:` | Lista os containers do projeto. Aqui há um só, a API. |
| `mlops-container:` | Nome do serviço. É o nome usado em comandos como `docker compose logs mlops-container`. |
| `container_name: mlops-container` | Nome fixo do container, para localizá-lo fácil com `docker ps`. |
| `build:` com `context: .` | Manda construir a imagem a partir da pasta atual (onde estão o código e o Dockerfile). |
| `dockerfile: Dockerfile` | Indica qual arquivo de receita usar na construção. |
| `ports:` com `"8000:8000"` | Liga a porta 8000 da sua máquina (esquerda) à porta 8000 do container (direita). Para abrir a API em `localhost:8001`, use `"8001:8000"`. |
| `environment:` | Define as variáveis de ambiente dentro do container: as mesmas quatro do seu `.env`. |
| `- WANDB_API_KEY=${WANDB_API_KEY}` | O `${...}` faz o Compose ler o valor no arquivo `.env` da pasta e repassá-lo ao container. Assim o |

| Linha | O que faz |
|---|---|
| | segredo fica no `.env` e não neste arquivo, que pode ir para o git. |
| `restart: unless-stopped` | Reinicia o container se ele cair ou se o Docker reiniciar, a menos que você o pare de propósito. |

Dentro do container não existe arquivo `.env` (ele não foi copiado para a imagem). Isso não é problema: o código lê as variáveis do ambiente, e o Compose já as entregou. As variáveis opcionais `MONGO_DB` e `MONGO_PRED_COLLECTION` usam o valor padrão do código; para mudá-las, acrescente-as à lista no mesmo formato.

### 4.5 Subir a API no container

1. Confira que o `.env` da raiz tem as quatro variáveis da Parte 2 e que a API local (`uvicorn`) está parada.
2. Na raiz do projeto, rode: `docker compose up --build`
3. Na primeira vez, o Docker baixa a imagem base e instala todas as dependências. Isso leva vários minutos e ocupa alguns GB de disco. Nas próximas, o cache torna o processo rápido.
4. Aguarde `Application startup complete.` nos logs. Como antes, a API baixa o seu modelo do W&B ao iniciar.
5. Abra http://localhost:8000/docs e repita o `POST /predictions` da Parte 3. O resultado esperado é o mesmo: código 200 e a predição do `confusion-clf`.
6. Em outro terminal, `docker compose ps` deve mostrar o `mlops-container` com status `running`.
7. Para encerrar, use Ctrl+C no terminal dos logs ou `docker compose down`.

Para a entrega, acompanhe os logs enquanto faz a requisição. Em um segundo terminal, rode `docker logs -f mlops-container` (equivale a `docker compose logs -f mlops-container`; acrescente `--tail 30` para mostrar só as últimas linhas). Com o acompanhamento aberto, repita o `POST /predictions` no Swagger. Aparecerá uma linha parecida com `"POST /predictions HTTP/1.1" 200 OK`. Faça o print do terminal mostrando essa linha. Ctrl+C encerra o acompanhamento sem parar o container.

Comandos do dia a dia:

| Comando | O que faz |
|---|---|
| `docker compose up --build` | Constrói a imagem (se algo mudou) e sobe o container, com os logs no terminal. |
| `docker compose up -d` | Sobe em segundo plano, sem prender o terminal. |

| Comando | O que faz |
|---|---|
| `docker compose logs -f mlops-container` | Acompanha os logs do container em tempo real. |
| `docker compose ps` | Lista os containers do projeto e o status de cada um. |
| `docker compose stop` | Para o container sem removê-lo. |
| `docker compose down` | Para e remove o container e a rede. A imagem continua guardada. |
| `docker compose build --no-cache` | Reconstrói a imagem do zero, ignorando o cache. |
| `docker images` | Lista as imagens guardadas na sua máquina. |

Problemas comuns no Docker:

| Sintoma | O que fazer |
|---|---|
| `port is already allocated` ou `address already in use` | Outra coisa usa a porta 8000, em geral o `uvicorn` local. Pare-o ou troque para `"8001:8000"` no compose. |
| `Cannot connect to the Docker daemon` | O Docker não está rodando. Abra o Docker Desktop (Windows e macOS) ou inicie o serviço do Docker (Linux). |
| `permission denied` em `docker.sock` (Linux) | Seu usuário não está no grupo `docker`. Rode `sudo usermod -aG docker $USER` e abra uma nova sessão. |
| Aviso `The "MONGO_URL" variable is not set` | O `.env` não está na mesma pasta do `docker-compose.yml`, ou o nome da variável está errado. |
| `docker compose ps` mostra `Restarting` | A API cai ao iniciar. Rode `docker compose logs mlops-container`: as causas são as da Parte 3 (artefato ou chave do W&B, `MONGO_URL`, IP no Atlas). |
| A instalação de dependências falha em Mac com chip Apple | Acrescente `platform: linux/amd64` ao serviço `mlops-container` no compose e refaça o `up --build`. Fica mais lento, pois emula outro processador. |

| Sintoma | O que fazer |
|---|---|
| `no space left on device` | Falta disco. Apague imagens antigas com `docker image prune` e tente de novo. |

## Entrega

Envie um arquivo, `lab_api_SEU_NOME.zip` ou `lab_api_SEU_NOME.pdf`, com os três itens abaixo.

| Item | O que deve conter | Como entregar |
|---|---|---|
| 1. Print do Swagger | A requisição `POST /predictions` com o seu nome no campo `nome`, o código 200 em *Server response* e o corpo da resposta com o resultado do modelo. | No ZIP: arquivo de imagem (`print_swagger.png`). No PDF: imagem inserida no documento. |
| 2. `controller.py` | O conteúdo completo do arquivo `src/modules/intent_classifier/controller.py` que você usou. | No ZIP: o próprio arquivo `controller.py`, sem alterar o nome. No PDF: o código colado como texto (não como imagem). |
| 3. Print dos logs do container | O terminal com o comando `docker logs -f mlops-container` mostrando a linha `POST /predictions` com `200 OK`, gerada pela sua requisição ao container (seguir a seção 4.5). | No ZIP: arquivo de imagem (`print_logs.png`). No PDF: imagem inserida no documento. |

Cuidados com o sigilo:

- O arquivo `.env` não vai na entrega e não pode estar no ZIP.
- O print não pode mostrar a chave do W&B nem a *connection string* do Mongo. Se o terminal aparecer no print, cubra qualquer trecho com essas informações.
- Se uma chave ou senha vazou (commit, print, mensagem), revogue-a imediatamente: no W&B em *User Settings, API Keys*; no Atlas, troque a senha do usuário do banco.

Prazo e canal de envio: a definir pelo professor.

## Checklist e dúvidas frequentes

Confira antes de enviar:

- ☐ O ambiente `mlops-2026-2` (Python 3.11) está ativo e o `pip install -r requirements.txt` terminou sem erro.
- ☐ O artefato `confusion-clf` tem `model.keras` e `model_config.yml` no W&B.
- ☐ O `.env` está na raiz do projeto e não aparece no `git status`.
- ☐ A API subiu (`Application startup complete.`) e o Swagger abriu em `/docs`.
- ☐ O print do Swagger mostra o código 200, o seu `nome` na resposta e nenhuma credencial.
- ☐ O container subiu com `docker compose up --build` e aparece como `running` em `docker compose ps`.
- ☐ O print dos logs mostra a linha `POST /predictions` com `200 OK` e nenhuma credencial.
- ☐ O `controller.py` enviado é o que você usou, completo.
- ☐ O arquivo final é um único `.zip` ou `.pdf` e não contém o `.env`.

Problemas comuns:

| Sintoma | O que fazer |
|---|---|
| `KeyError: 'MONGO_URL'` | O `.env` não está na raiz ou o `uvicorn` foi rodado de outra pasta. |
| `ServerSelectionTimeoutError` | Libere o seu IP em **Network Access** no Atlas e espere cerca de um minuto. |
| `Authentication failed` | Confira usuário e senha na `MONGO_URL` (sem caracteres especiais). |
| `Could not fetch artifact` | O caminho em `WANDB_MODELS` está errado ou a `WANDB_API_KEY` é inválida. |
| `Config file (_config.yml) not found` | O artefato não tem os dois arquivos. Retreine o modelo no Colab. |
| `Unrecognized keyword arguments ... quantization_config` | Versões diferentes do Keras no Colab e na sua máquina. Peça orientação ao professor. |
| `422 Unprocessable Entity` | Envie os três campos: `id`, `nome` e `text`. |
| `Address already in use` | Use outra porta: `--port 8001`. |